PYTHIA: Universal Question Answering over Knowledge Graphs

Sergios-Anestis Kefalidis[1], Konstantinos Plas[1,2] and Manolis Koubarakis[1,2]

[1]National and Kapodistrian University of Athens, Athens, Greece
[2]Athena Research Centre, Marousi, Greece

Graph Reasoning

Agentic Design

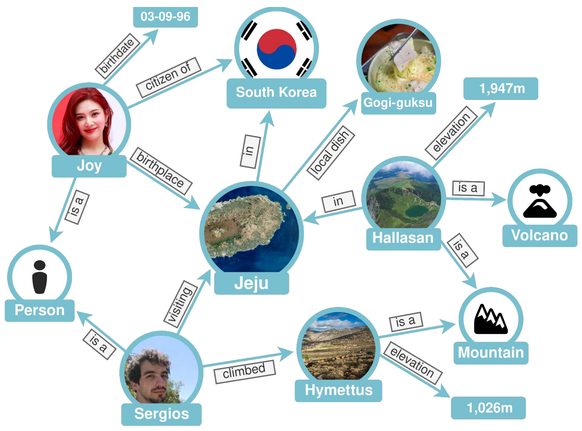
Text-to-SPARQL

Question Answering

Read the Paper

1. Motivation & Background

Knowledge Graphs (KGs) model data as *nodes* and *relations* between them.
Useful to both **humans** and **LLMs (KG-RAG)**.



Usability barriers for users

- **Know** SPARQL!
- **Know** the target KG's ontology and quirks!
- **Search** for topic entities!

Usability barriers for LLMs

- Most KGs are too big to be given as context in their entirety!
- **Which information is relevant?**

Knowledge Graph Question Answering (KGQA) provides a **natural language interface** to query information from a **KG**, thus improving accessibility.

Major Challenges

01 Training-Free
Requiring training data severely limits viability in real-world, purpose-built KGs.

02 Transferability
The ability to seamlessly deploy across different KGs without modifications.

03 Generalization
Answering unseen questions outside the scope of validation.

Prior Attempts and Limitations

- **LLMs** have facilitated significant progress in KGQA, enabling ***training-free*** and ***transferable*** methods.
- **LLM-guided exploration:** Such systems explore the KG iteratively, without requiring prior knowledge. However they underperform in multi-hop and multi-entity questions.
- **Agentic Architectures:** Transitioning from the traditional pipeline architectures to ReAct-like agents enables reflection and adaptability. However, the core LLM is responsible for successfully orchestrating multiple steps to solve a problem.

2. Methodology

A Novel Framing of the KGQA Task

Any **Agent** consists of two parts: **i)** the backbone **LLM** guided by a **prompt** and **ii)** the **tools** at its disposal. It follows, that choosing an effective framing for the task is a crucial step in designing any agentic system.

However, prior work has generally overlooked this aspect, focusing instead on just tooling.

Limited Description [1]
Provides generic task instructions with only examples as guidance.

Prompt:
You need to understand a question, and plan for the process of constructing a query to solve this question.

Examples follow...

Simple Iteration [2,3]
Describes iterative SPARQL construction by finding entities and properties.

Prompt (paraphrased):
You are a KGQA assistant. Your task is to understand the question, find entities and properties in the KG related to it and iteratively build up a SPARQL query.

Exploration-Centric Framing (Ours)
Describing KGQA as an exploration task between nodes and their relations:

i) Entity Linking
Identify named entities & link to KG nodes

ii) Property Retrieval
Retrieve properties of grounded nodes

iii) Semantic Connection Discovery
Explore relations between nodes:

- **Known-to-Unknown**
- **Known-to-Known**

Result A mental model for entity linking and exploration, **abstracting low-level details**.

[1] *QueryAgent: A Reliable and Efficient Reasoning Framework with Environmental Feedback-based Self-Correction*. Huang et al. ACL 2024.
[2] *SPINACH: SPARQL-Based Information Navigation for Challenging Real-World Questions*. Liu et al. Findings of EMNLP 2024
[3] *GRASP: Generic Reasoning And SPARQL Generation across Knowledge Graphs*. Walter et al. ISWC 2025.

Operationalizing the Framing through Tools

Each tool of **PYTHIA** is tailored to a specific aspect of the conceptual model. Despite their powerful exploration capabilities, they have simple interfaces that abstract their complexity (and use separate LLM contexts).

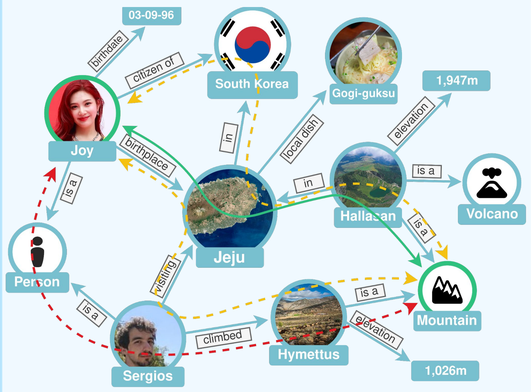
Known-to-Known Connection Discovery

Tool: graph_search

Q: "What mountains exist in Joy's birthplace?"

Step 1) Entity Linking (Find topic entities)
retrieve_entities_and_classes("Joy")
retrieve_entities_and_classes("Mountain")

Step 2) Connection Discovery (Known-to-Known)
graph_search(kg:Joy, kg:Mountain, Question)



How graph_search works internally:

i) Bidirectional Search: Find paths connecting the two target nodes in the KG.

ii) Cosine Filtering: Prune paths by calculating cosine similarity against the query string (**red paths**).

iii) LLM Selection: LLM evaluates which paths best describe the query and those are selected for retrieval (**green paths**).

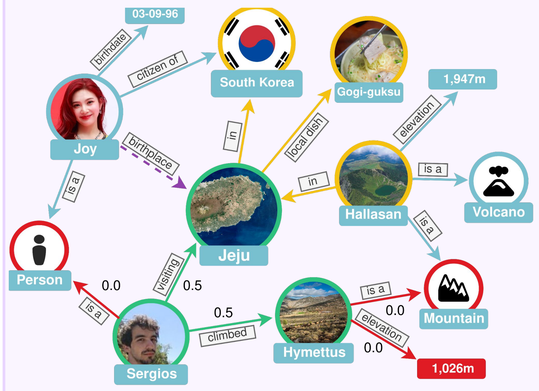
Known-to-Unknown Connection Discovery

Tool: beam_search

Q: "What local food can I (Sergios) try?"

Step 1) Entity Linking (Find topic entities)
retrieve_entities_and_classes("Sergios")

Step 2) Connection Discovery (Known-to-Unknown)
beam_search(kg:Sergios, "local dish for Sergios")



How beam_search works internally:

i) Expand Frontier: Expand predicates of current frontier nodes, keeping only the top ***W*** options via weighted pruning.

ii) LLM Decision: Candidate paths are evaluated by the LLM to decide the most promising direction to continue search.

iii) Frontier Update: The active frontier is updated, grouping same-predicate nodes into "super-nodes".

Additional tools not shown: *get_predicates_for_node, execute_query, use_last_query_as_answer*

3. Results

Table 1. Evaluation across multiple KGs (Macro-F1).

System	WebQSP	CWQ	QALD-9	LC-QuAD	QALD-10	BESTIARY
Zero-shot (or static ICL)						
LLM-Inherent	36.42	27.99	36.27	28.23	35.13	15.78
ToG	35.33	32.12	40.57	41.81	40.36	N/A
ToG*	43.76	36.22	42.40	46.55	41.87	19.93
PoG*	51.83	15.41	24.74	12.48	21.50	6.00
PoG* (Adjusted)	56.65	43.88	60.37	57.86	55.59	11.53
GRASP	65.79	44.10	60.88	44.39	64.98	42.66
GRASP*	64.36	44.40	64.39	48.65	66.38	41.38
PYTHIA	66.71	53.57	68.08	59.34	74.40	51.01
PYTHIA*	**73.99**	**61.62**	**73.08**	**67.10**	**76.11**	**51.99**
Few-shot (Dynamic)						
LLM-SPARQL	28.19	27.69	7.11	31.07	2.14	N/A
LLM-SPARQL*	34.55	32.21	8.73	31.26	1.75	N/A
GRASP	71.54	59.20	55.81	55.37	60.62	N/A
GRASP*	76.44	65.60	66.27	64.92	70.57	N/A
PYTHIA	75.41	70.12	67.41	70.34	75.59	N/A
PYTHIA*	**80.69**	**75.14**	**76.82**	**76.94**	**77.63**	N/A

Figures 1 and 2. Analysis of multi-fact & multi-hop performance.

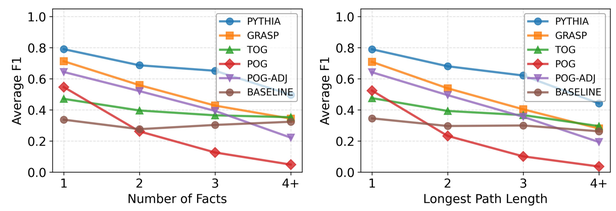
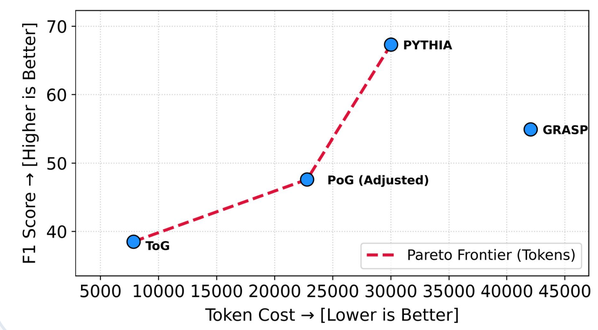


Figure 3. Token efficiency.



4. Conclusions

PYTHIA: SOTA Zero-Shot & Transferable KGQA
A KG-agnostic QA engine achieving new state-of-the-art performance & efficiency in its class.

Novel Conceptual Model
Relationship-driven exploration framing that abstracts complex KG exploration.

Powerful Tooling that Abstracts Complexity
Custom LLM-guided beam search & Graph Search with LLM Selection to reduce the "mental" load of the main KGQA Agent.

Robust Evaluation & Open Source
Refined prior models to facilitate the most robust evaluation of zero-shot KGQA systems to date. Code & data open!

HELLENIC REPUBLIC
National and Kapodistrian University of Athens

ATHENA Research & Innovation Information Technologies

August 9–13, 2026
KDD2026
Jeju, Korea